



Explore | Expand | Enrich

<Codemithra />TM

SEGREGATE EVEN AND ODD NODES IN A LINKED LIST



Test time on Sort the bitonic in Doubly LL

Url: <https://forms.gle/3BybjYD1x4CNBmC78>



SEGREGATE EVEN AND ODD NODES IN A LINKED LIST

EXPLANATION

- To segregate even and odd nodes in a singly linked list, you can create two separate linked list, you can create two separate linked list: one for even nodes and one for odd nodes.
- Then traverse the original list moving nodes to their respective new lists based on whether their values are even or odd. Finally merge the two list to obtain the final segregated linked list.

SEGREGATE EVEN AND ODD NODES IN A LINKED LIST

EXPLANATION

Algorithm

- The original list now contains only even nodes, and the odd list contains only odd nodes.
- After traversing the entire original list, connect the last node of the even list to the first node of the odd list to join them.
- Return the head of the original list (which contains even nodes).

SEGREGATE EVEN AND ODD NODES IN A LINKED LIST

Example 1:

Input Linked List:



Output Linked List:



SEGREGATE EVEN AND ODD NODES IN A LINKED LIST

EXPLANATION

- In our original linked list, we have numbers: $1 \rightarrow 2 \rightarrow 3$.
- We want to separate even numbers from odd numbers and keep the order the same.
- So, first, we find the even number, which is 2. We keep it at the beginning.
- Next, we find the odd numbers, which are 1 and 3. We keep their order and put them after 2.
- Now, our new linked list is segregated: $2 \rightarrow 1 \rightarrow 3$.



SEGREGATE EVEN AND ODD NODES IN A LINKED LIST

Example 2:

2 → 1 → 6 → 4 → 8.

Input linked list



Output linked list



SEGREGATE EVEN AND ODD NODES IN A LINKED LIST

Example 2:

$2 \rightarrow 1 \rightarrow 6 \rightarrow 4 \rightarrow 8$.

In our original linked list, we have numbers: $2 \rightarrow 1 \rightarrow 6 \rightarrow 4 \rightarrow 8$.

We want to separate even numbers from odd numbers and keep the order the same.

So, first, we find the even numbers, which are 2, 6, 4, and 8. We keep them at the beginning in the same order, forming $2 \rightarrow 6 \rightarrow 4 \rightarrow 8$.

Next, we find the odd number, which is 1. We keep it after the even numbers, maintaining its original order, forming $2 \rightarrow 6 \rightarrow 4 \rightarrow 8 \rightarrow 1$.



SEGREGATE EVEN AND ODD NODES IN A LINKED LIST

EXPLANATION

Input:

Original list:

1 2 3 4 5

Output:

Segregated list (even
before odd):

2 4 1 3 5



SEGREGATE EVEN AND ODD NODES IN A LINKED LIST

```

class Node {
    int data;
    Node next;
    public Node(int data) {
        this.data = data;
        this.next = null;
    }
}

class LinkedList {
    Node head;

    public void append(int data) {
        Node newNode = new Node(data);
        if (head == null) {
            head = newNode;
            return;
        }
        Node current = head;
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode;
    }

    public void segregateEvenOdd() {
        if (head == null) {
            System.out.println("The list is empty.");
            return;
        }
        Node evenHead = null, evenTail = null;
        Node oddHead = null, oddTail = null;
        Node current = head;
        while (current != null) {
            int data = current.data;
            if (data % 2 == 0) { // even node
                if (evenHead == null) {
                    evenHead = evenTail = current;
                }
            }
        }
    }
}

```



SEGREGATE EVEN AND ODD NODES IN A LINKED LIST

```

        } else {
            evenTail.next = current;
            evenTail = current;
        }
    } else { // odd node
        if (oddHead == null) {
            oddHead = oddTail = current;
        } else {
            oddTail.next = current;
            oddTail = current;
        }
    }
    current = current.next;
}
// Join even and odd lists
if (evenHead != null) {
    evenTail.next = oddHead;
}

```

```

if (oddHead != null) {
    oddTail.next = null;
}
head = evenHead != null ? evenHead : oddHead;
}

public void printList() {
    Node current = head;
    while (current != null) {
        System.out.print(current.data + " ");
        current = current.next;
    }
    System.out.println();
}
}

```



SEGREGATE EVEN AND ODD NODES IN A LINKED LIST

```
public class Main {  
    public static void main(String[] args) {  
        LinkedList list = new LinkedList();  
        list.append(1);  
        list.append(2);  
        list.append(3);  
        list.append(4);  
        list.append(5);  
  
        System.out.println("Original list:");  
        list.printList();  
  
        list.seggregateEvenOdd();  
  
        System.out.println("Segregated list (even before odd):");  
        list.printList();  
    }  
}
```

SEGREGATE EVEN AND ODD NODES IN A LINKED LIST

Time complexity and space complexity

Time complexity: $O(N)$, where N is the number of nodes in the linked list.

Space complexity: $O(1)$, constant space is used for the pointers and temporary variables.

INTERVIEW QUESTIONS

1. How would you segregate even and odd nodes in a singly linked list while maintaining the order of nodes?

Answer: To segregate even and odd nodes while preserving the order, you can iterate through the linked list, move even nodes to a separate even list, and odd nodes to an odd list. Finally, merge the two lists while keeping the order.

INTERVIEW QUESTIONS

2. What is the time complexity of segregating even and odd nodes in a singly linked list using the above approach?

Answer: The time complexity is $O(n)$, where n is the number of nodes in the linked list, as we traverse the list once to segregate the nodes.

INTERVIEW QUESTIONS

3. Can you solve this problem in-place without using extra space for additional lists?

Answer: Yes, an in-place solution is possible. You can maintain two pointers, one for even nodes and one for odd nodes, while rearranging the nodes within the existing list.

INTERVIEW QUESTIONS

4. What is the space complexity of the in-place solution for segregating even and odd nodes?

Answer: The in-place solution has a space complexity of $O(1)$, as it does not require additional data structures.

INTERVIEW QUESTIONS

5. How can you modify the linked list to place all even nodes before all odd nodes, regardless of the original order?

Answer: To place all even nodes before odd nodes, you can create two separate lists for even and odd nodes, then concatenate the odd list to the end of the even list.

INTERVIEW QUESTIONS

6.If the linked list nodes contain values, and you need to maintain the order of nodes based on their values while segregating even and odd nodes, how would you modify your approach?

Answer: In this case, you would need to compare node values and reorder nodes accordingly while segregating even and odd nodes, ensuring that the relative order of values is preserved in both the even and odd portions of the list.



<https://learn.codemithra.com>



THANK YOU



+91 78150 95095



codemithra@ethnus.com



www.codemithra.com